

MODULE 29

Getting Started with Retrievers, RAG, OpenAI & Ollama

Lectures Covered

- Lecture 1 — Getting Started with Retrievers
- Lecture 2 — Wikipedia Retriever & Vector Store Retriever
- Lecture 3 — Maximal Marginal Relevance (MMR) Retriever
- Lecture 4 — Multi-Query & Contextual Compression Retriever
- Lecture 5 — What is RAG (Shifting From Fine-Tuning to RAG)
- Lecture 6 — YouTube Content Chatbot (RAG Application)
- Lecture 7 — Simple ChatBot with Ollama and LangSmith Tracking

Lecture 1: Getting Started with Retrievers

A Retriever is a LangChain interface that takes a query string and returns a list of relevant Documents. Retrievers are the bridge between a user's question and the knowledge stored in a vector store or external source.

What is a Retriever?

Retriever interface

Input: a query string

Output: List[Document] — ranked by relevance

All retrievers implement: `retriever.invoke('your question') -> [Document, ...]`

Retrievers are composable — plug into any LCEL chain as a component.

BaseRetriever — Core API

```
# Any retriever follows the same interface
from langchain_core.retrievers import BaseRetriever

# Invoke a retriever
docs = retriever.invoke('What is gradient descent?')
for doc in docs:
    print(doc.page_content[:200])
    print(doc.metadata)

# Retrievers integrate seamlessly into LCEL chains
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

rag_chain = (
    {'context': retriever, 'question': RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)
answer = rag_chain.invoke('Explain backpropagation')
```

Types of Retrievers in LangChain

Retriever	Source Best For
VectorStoreRetriever	FAISS / Chroma / Pinecone Semantic similarity search over documents
WikipediaRetriever	Wikipedia API Quick factual Q&A without pre-loading docs
MMR Retriever	Vector store with diversity Avoiding redundant retrieved chunks
MultiQueryRetriever	LLM + vector store Complex queries that benefit from rephrasing
ContextualCompressionRetriever	LLM + any retriever Extracting only relevant sentences from chunks
SelfQueryRetriever	LLM + vector store with metadata Queries with filters ('find docs from 2023')
ArxivRetriever	Arxiv API Scientific paper retrieval

Lecture 2: Wikipedia Retriever & Vector Store Retriever

Wikipedia Retriever

Fetches Wikipedia articles on the fly for any query — no pre-loading or vector store needed. Great for general knowledge Q&A applications.

```
pip install wikipedia
```

```

from langchain_community.retrievers import WikipediaRetriever

retriever = WikipediaRetriever(
    top_k_results = 3,      # number of Wikipedia articles to fetch
    doc_content_chars_max = 2000 # truncate article content
)

docs = retriever.invoke('Transformer neural network architecture')
for doc in docs:
    print(doc.metadata['title'])
    print(doc.page_content[:300])
    print('---')

```

Vector Store Retriever

The most common retriever — performs embedding-based similarity search over a pre-built vector store (FAISS or ChromaDB).

```

from langchain_community.vectorstores import FAISS, Chroma
from langchain_openai import OpenAIEmbeddings

embed = OpenAIEmbeddings(model='text-embedding-3-small')

# Build vector store from documents
db = FAISS.from_documents(chunks, embed)

# — Basic similarity retriever —————
retriever = db.as_retriever(
    search_type = 'similarity', # default
    search_kwargs = {'k': 4}    # return top 4 chunks
)

# — Similarity with score threshold —————
retriever = db.as_retriever(
    search_type = 'similarity_score_threshold',
    search_kwargs = {'score_threshold': 0.75, 'k': 6}
)

# Invoke
results = retriever.invoke('What is attention mechanism?')
print(f'Retrieved {len(results)} chunks')

```

Lecture 3: Maximal Marginal Relevance (MMR) Retriever

Standard similarity search returns the top-k most similar chunks — but these can be nearly identical, providing redundant context. MMR balances relevance with diversity to return a varied set of useful chunks.

How MMR Works

MMR formula at each selection step

Score = $\lambda * \text{similarity}(\text{chunk}, \text{query}) - (1 - \lambda) * \text{max_similarity}(\text{chunk}, \text{already_selected})$

$\lambda = 1.0$ -> pure similarity (same as standard search)

$\lambda = 0.0$ -> pure diversity (maximally different chunks)

$\lambda = 0.5$ -> balanced: relevant but not repetitive (recommended)

```

from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings

db = FAISS.from_documents(chunks, OpenAIEmbeddings())

```

```
# MMR retriever
mmr_retriever = db.as_retriever(
    search_type = 'mmr',
    search_kwargs = {
        'k': 4,      # final chunks to return
        'fetch_k': 20, # candidate pool size before MMR filtering
        'lambda_mult': 0.5 # diversity balance (0=diverse, 1=relevant)
    }
)

docs = mmr_retriever.invoke('Explain neural network training')
# Returns 4 relevant but DIVERSE chunks – less repetition in context

# Comparison
sim_docs = db.similarity_search('Explain neural network training', k=4)
# sim_docs may contain near-duplicate paragraphs
# mmr_docs will be more varied in content
```

When to use MMR

Use MMR when documents have many similar/overlapping paragraphs (e.g. books, reports).

Standard similarity is fine for diverse document collections (e.g. mixed web pages).

fetch_k should be 3-5x larger than k — MMR needs a pool to select from.

lambda_mult=0.5 is the standard starting point; tune based on answer quality.

Lecture 4: Multi-Query & Contextual Compression Retriever

MultiQueryRetriever — LLM-Powered Query Expansion

Complex user queries can fail similarity search if phrased differently from the indexed documents.

MultiQueryRetriever uses an LLM to automatically generate multiple rephrasings of the query, retrieves chunks for each, then deduplicates the results.

```
from langchain.retrievers import MultiQueryRetriever
from langchain.openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o', temperature=0)
base_ret = db.as_retriever(search_kwargs={'k': 3})

mq_retriever = MultiQueryRetriever.from_llm(
    retriever = base_ret,
    llm = llm
)

# For query: 'How do neural networks learn?'
# LLM generates variants like:
# 'What is the training process of a neural network?'
# 'Explain backpropagation and gradient descent'
# 'How do weights get updated in deep learning?'
# Retrieves chunks for each -> deduplicates -> returns union

import logging
logging.getLogger('langchain.retrievers.multi_query').setLevel(logging.INFO)
# Logs the generated queries for inspection

docs = mq_retriever.invoke('How do neural networks learn?')
print(f'Retrieved {len(docs)} unique chunks')
```

ContextualCompressionRetriever

Retrieved chunks often contain irrelevant sentences surrounding the actual answer. ContextualCompressionRetriever uses an LLM to compress each retrieved chunk — keeping only the sentences relevant to the query.

```

from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o', temperature=0)
compressor = LLMChainExtractor.from_llm(llm)

cc_retriever = ContextualCompressionRetriever(
    base_compressor = compressor,
    base_retriever = db.as_retriever(search_kwargs={'k': 4})
)

# Original chunk might be 500 tokens with 1 relevant sentence
# After compression: only the relevant sentences are returned
compressed_docs = cc_retriever.invoke('What activation function does GPT use?')
for doc in compressed_docs:
    print(doc.page_content) # concise, directly relevant content

# Combine MMR + Compression for best results
mmr_ret = db.as_retriever(search_type='mmr', search_kwargs={'k':4, 'fetch_k':20})
best_ret = ContextualCompressionRetriever(
    base_compressor=LLMChainExtractor.from_llm(llm),
    base_retriever=mmr_ret
)

```

Retriever	Advantage Cost
VectorStoreRetriever	Fast, simple, no extra LLM call May miss paraphrased queries
MMR Retriever	Diverse, non-redundant chunks Slightly slower than similarity
MultiQueryRetriever	Catches varied phrasings Extra LLM call per query (3-5 variants)
ContextualCompression	Precise, focused answers Extra LLM call per retrieved chunk

Lecture 5: What is RAG — Shifting from Fine-Tuning to RAG

RAG (Retrieval-Augmented Generation) is an architecture that combines a retrieval system with an LLM at inference time — giving the model access to external knowledge without modifying its weights.

Fine-Tuning vs RAG

Dimension	Fine-Tuning vs RAG
How it works	Fine-Tune: train model weights on new data RAG: retrieve docs at query time, inject into prompt
Knowledge update	Fine-Tune: requires retraining (hours/days) RAG: update vector store in minutes
Cost	Fine-Tune: high GPU compute RAG: low — only embedding + retrieval cost
Data needed	Fine-Tune: hundreds to thousands of labelled examples RAG: any corpus of documents
Hallucination	Fine-Tune: can still hallucinate RAG: answers grounded in retrieved docs
Best for	Fine-Tune: model style/behaviour/task RAG: adding factual knowledge from documents
Transparency	Fine-Tune: hard to trace knowledge source RAG: show retrieved source chunks

RAG Architecture

RAG Pipeline — Two Phases

INDEXING (offline, once):

1. Load documents (DocumentLoader)
2. Split into chunks (TextSplitter, chunk_size=1000, overlap=200)
3. Embed chunks (OpenAIEmbeddings / HuggingFaceEmbeddings)
4. Store in vector DB (FAISS / ChromaDB)

RETRIEVAL + GENERATION (online, every query):

5. Embed user query
6. Similarity search -> top-k relevant chunks
7. Format chunks as context in prompt
8. LLM generates answer grounded in retrieved context

When to Choose RAG over Fine-Tuning

- **Use RAG when:** you have proprietary docs, need up-to-date info, want source citations, or have limited data.
- **Use Fine-Tuning when:** you need to change the model's tone/style/format, teach a specialised skill, or compress knowledge permanently into weights.
- **Use Both:** Fine-tune for style + RAG for knowledge — the production standard for enterprise LLM apps.

Lecture 6: YouTube Content Chatbot (RAG Application)

A complete RAG application that loads a YouTube video transcript, indexes it, and answers questions about the video content using an LLM — all with LangChain.

Full YouTube RAG Pipeline

```

pip install langchain langchain-openai langchain-community
pip install youtube-transcript-api pytube chromadb

from langchain_community.document_loaders import YoutubeLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

# — Step 1: Load YouTube transcript —————
loader = YoutubeLoader.from_youtube_url(
    'https://www.youtube.com/watch?v=dQw4w9WgXcQ',
    add_video_info = True, # includes title, author in metadata
    language = ['en']
)
docs = loader.load()
print(f'Transcript length: {len(docs[0].page_content)} chars')

# — Step 2: Split transcript —————
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200
)
chunks = splitter.split_documents(docs)
print(f'Chunks: {len(chunks)}')

# — Step 3: Embed and store —————
embed = OpenAIEmbeddings(model='text-embedding-3-small')
db = Chroma.from_documents(chunks, embed,
                          persist_directory='yt_chroma')

# — Step 4: Build RAG chain —————
retriever = db.as_retriever(search_kwargs={'k': 4})
llm = ChatOpenAI(model='gpt-4o', temperature=0)

```

```

prompt = ChatPromptTemplate.from_template('''
You are an assistant answering questions about a YouTube video.
Use ONLY the context below. If unsure, say you don't know.

Context: {context}
Question: {question}
Answer:''')

def format_docs(docs):
    return '\n\n'.join(d.page_content for d in docs)

rag_chain = (
    {'context': retriever | format_docs, 'question': RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)

# — Step 5: Query —————
answer = rag_chain.invoke('What are the main topics discussed?')
print(answer)

```

YouTube RAG tips

Add `add_video_info=True` to store title, author, `publish_date` in metadata.
 For long videos (2h+) use `chunk_size=500` to avoid losing detail at boundaries.
 Use MMR retriever to avoid retrieving near-duplicate transcript segments.
 Persist ChromaDB so you don't re-embed the same video on every run.

Lecture 7: Simple ChatBot with Ollama and LangSmith Tracking

Build a local conversational chatbot using Ollama (no API cost), with conversation memory, and full observability via LangSmith tracing.

Setup — Ollama Local LLM

```

# 1. Install Ollama: https://ollama.com
ollama pull llama3          # download LLaMA 3 8B locally
ollama serve                # starts local server at localhost:11434

# 2. Install Python packages
pip install langchain langchain-community langchain-ollama
pip install python-dotenv streamlit

# 3. LangSmith setup (.env file)
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=ls__your_key_here
LANGCHAIN_PROJECT=ollama-chatbot

```

Complete Chatbot with Memory

```

from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()  # loads LangSmith env vars -> tracing auto-enabled

# — Model —————
llm = ChatOllama(model='llama3', temperature=0.7)

```

```
# — Prompt with memory placeholder —————
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant. Be concise and clear.'),
    MessagesPlaceholder(variable_name='history'),
    ('human', '{input}'))

# — Chain —————
chain = prompt | llm | StrOutputParser()

# — Session memory store —————
store = {}
def get_session(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

chatbot = RunnableWithMessageHistory(
    chain,
    get_session,
    input_messages_key = 'input',
    history_messages_key = 'history'
)

# — Conversation loop —————
config = {'configurable': {'session_id': 'user_001'}}

while True:
    user_input = input('You: ')
    if user_input.lower() in ['quit', 'exit']: break
    response = chatbot.invoke({'input': user_input}, config=config)
    print(f'Bot: {response}')
```

LangSmith Tracing — What You See

LangSmith View	Information Shown
Run tree	Every step: ChatPromptTemplate -> ChatOllama -> StrOutputParser
Inputs/Outputs	Exact prompt sent, exact response received at each step
Latency	Time taken per step and total chain duration
Token usage	Prompt tokens + completion tokens per LLM call
History	Full conversation history passed to the model each turn
Errors	Full stack trace for any failed run

Streamlit UI for the Chatbot

```
# app.py — Streamlit chat interface
import streamlit as st
from chatbot import chatbot # import your chain with memory

st.title('Local AI Chatbot (LLaMA 3 via Ollama)')

# Initialise session state
if 'messages' not in st.session_state:
    st.session_state.messages = []

# Display chat history
for msg in st.session_state.messages:
    with st.chat_message(msg['role']):
        st.write(msg['content'])

# Handle new input
```



```
if prompt := st.chat_input('Ask me anything...'):
    st.session_state.messages.append({'role': 'user', 'content': prompt})
    with st.chat_message('user'): st.write(prompt)

    config = {'configurable': {'session_id': 'streamlit_user'}}
    response = chatbot.invoke({'input': prompt}, config=config)

    st.session_state.messages.append({'role': 'assistant', 'content': response})
    with st.chat_message('assistant'): st.write(response)

# Run: streamlit run app.py
```

Ollama memory requirement

LLaMA 3 8B requires ~5 GB RAM minimum (8 GB recommended).

LLaMA 3 70B requires ~40 GB RAM (use quantised Q4 version ~20 GB).

For machines with < 8 GB RAM: use phi3 (3.8B, ~2.5 GB) or gemma:2b.

Check: `ollama list` -- shows downloaded models and their sizes.

End of Module 29 Notes — Generative AI Course